

SBI 811: Computer Programming in Bioinformatics

Lecture 4 — Conditions, Loops and Collections

Duration: 3 hours

Level: Complete beginners

Prerequisite: Lecture 3 — variables, data types, strings, operators, indexing, slicing, sequence length and GC percentage

Biological applications: sequence validation, GC classification, motif searching and processing several sequences

The central question

Lecture 3 programs performed each instruction once. In this lecture, programs become more useful because they can:

1. **make decisions** using conditions;
2. **repeat instructions** using loops; and
3. **store several related values** using lists and tuples.

Run the notebook from top to bottom. Later cells use variables created earlier. If results become confusing, select **Kernel** → **Restart Kernel and Run All Cells**.

Learning outcomes

By the end of this lecture, you should be able to:

1. distinguish assignment (`=`) from comparison (`==`);
2. use comparison operators to create Boolean results;
3. combine conditions using `and` , `or` and `not` ;
4. write correctly indented `if` , `elif` and `else` blocks;
5. use conditions to validate and classify biological data;
6. explain why loops are needed;
7. use a `for` loop to process a string and a list;
8. use `range()` to generate a sequence of integers;
9. use `enumerate()` when both a position and a value are required;
10. use a controlled `while` loop;
11. create, access and modify lists;
12. append, insert and remove list elements;
13. distinguish mutable lists from immutable tuples; and
14. combine collections, conditions and loops in a small sequence-analysis workflow.

Functions, dictionaries, file reading and pandas are intentionally postponed.

Suggested three-hour lesson plan

Time	Activity
0–40 min	Comparisons, Boolean logic and <code>if</code> statements
40–70 min	Instructor demonstration: validating and classifying one sequence
70–115 min	Lists and tuples: storing and modifying collections
115–145 min	<code>for</code> , <code>range()</code> , <code>enumerate()</code> and a controlled <code>while</code> loop
145–170 min	Integrated biological practical: process several sequences
170–180 min	Debugging and formative review

Part 1 — From calculations to decisions

Lecture 3 calculated GC percentage:

```
gc_percentage = (gc_count / sequence_length) * 100
```

But a useful program may need to decide what the result means:

```
If GC percentage is at least 60, report high GC.  
Otherwise, report moderate or low GC.
```

This requires a **condition**: an expression that Python evaluates as either `True` or `False`.

Part 2 — Boolean values and comparisons

2.1 Boolean values

The Boolean type has only two possible values:

```
True  
False
```

The capital letters matter. `true` and `false` are not Python Boolean values.

```
In [ ]: passed_quality_check = True  
contains_unknown_bases = False  
  
print(passed_quality_check)  
print(contains_unknown_bases)  
print(type(passed_quality_check))
```

2.2 Comparison operators

Comparison operators compare values and return `True` or `False`.

Operator	Meaning	Example
<code>==</code>	equal to	<code>length == 100</code>
<code>!=</code>	not equal to	<code>base != "N"</code>
<code>></code>	greater than	<code>gc > 60</code>
<code><</code>	less than	<code>length < 50</code>
<code>>=</code>	greater than or equal to	<code>quality >= 30</code>
<code><=</code>	less than or equal to	<code>n_count <= 2</code>

```
In [ ]: sequence_length = 24

print(sequence_length == 24)
print(sequence_length != 24)
print(sequence_length > 20)
print(sequence_length < 20)
print(sequence_length >= 24)
print(sequence_length <= 23)
```

2.3 Assignment and comparison are different

`sequence_length = 24`

The single equals sign **assigns** the value 24.

`sequence_length == 24`

The double equals sign **compares** the current value with 24.

Read `==` as “is equal to?”

Using `=` where Python expects a comparison is a very common beginner mistake.

Guided practice 1 — Predict the Boolean result

Predict each result before running the cell.

```
In [ ]: gc_percentage = 55.0
sample_name = "sample_A"

print(gc_percentage >= 50)
print(gc_percentage > 60)
print(sample_name == "sample_A")
print(sample_name != "sample_B")
```

2.4 Comparing strings

String comparison is case-sensitive.

```
In [ ]: base = "A"

print(base == "A")
print(base == "a")
```

```
print("ATGC" == "ATGC")
print("ATGC" == "atgc")
```

Standardizing sequence text with `.upper()` before comparison reduces unexpected differences.

2.5 Membership with `in` and `not in`

The `in` operator asks whether a value occurs inside another value or collection.

```
In [ ]: dna = "ATGCGTAA"

print("ATG" in dna)
print("NNN" in dna)
print("N" not in dna)
```

These expressions mean:

- Does the motif `ATG` occur in the sequence?
- Does `NNN` occur in the sequence?
- Is `N` absent from the sequence?

Membership tests are comparisons and therefore return Boolean values.

Part 3 — Combining conditions

3.1 The `and` operator

`and` produces `True` only when **both** conditions are true.

Example biological rule:

A sequence passes when its length is at least 20 and it contains no N.

```
In [ ]: dna = "ATGCGTAACTGGCTAACGTA"

length_is_sufficient = len(dna) >= 20
has_no_unknown_bases = "N" not in dna

passes = length_is_sufficient and has_no_unknown_bases

print("Length sufficient:", length_is_sufficient)
print("No unknown bases:", has_no_unknown_bases)
print("Passes both checks:", passes)
```

3.2 The `or` operator

`or` produces `True` when **at least one** condition is true.

Example rule:

A nucleotide is a purine when it is A or G.

```
In [ ]: base = "G"
is_purine = base == "A" or base == "G"

print(is_purine)
```

The membership form is shorter and often clearer:

```
is_purine = base in "AG"
```

```
In [ ]: base = "G"
is_purine = base in "AG"

print(is_purine)
```

3.3 The `not` operator

`not` reverses a Boolean result.

```
In [ ]: contains_n = "N" in "ATGCNTA"

print("Contains N:", contains_n)
print("Does not contain N:", not contains_n)
```

3.4 Truth tables in plain language

A	B	A and B	A or B
True	True	True	True
True	False	False	True
False	True	False	True
False	False	False	False

- `and` is strict: every required condition must pass.
- `or` is flexible: one passing condition is enough.
- `not` reverses the answer.

Guided practice 2 — Combine quality checks

A sample passes when:

- read count is at least 1,000; **and**
- ambiguous bases are no more than 2.

Complete or modify the values and predict the result.

```
In [ ]: read_count = 1500
ambiguous_bases = 1

passes_quality = read_count >= 1000 and ambiguous_bases <= 2
```

```
print("Passes quality checks:", passes_quality)
```

Part 4 — Making decisions with `if`

4.1 A simple `if` statement

```
if condition:  
    instruction
```

Python executes the indented instruction only when the condition is `True`.

```
In [ ]: dna = "ATGCGTAA"  
  
if "ATG" in dna:  
    print("Start motif found")
```

Important features:

1. `if` begins the decision.
2. The condition follows `if`.
3. A colon `:` ends the `if` line.
4. The controlled instruction is indented.

If the condition is false, the indented block is skipped.

```
In [ ]: dna = "CCGCGTAA"  
  
if "ATG" in dna:  
    print("Start motif found")  
  
print("Sequence check completed")
```

`Sequence check completed` is not indented, so it runs regardless of the condition.

4.2 Indentation defines a block

Python commonly uses four spaces for each indentation level.

Correct:

```
if gc_percentage >= 50:  
    print("GC is at least 50%")
```

Incorrect:

```
if gc_percentage >= 50:  
print("GC is at least 50%")
```

The second example causes an `IndentationError` because Python expects an indented block.

4.3 if and else

`else` provides an alternative when the condition is false.

```
In [ ]: dna = "ATGCNTAA"

if "N" not in dna:
    print("Sequence contains only identified bases")
else:
    print("Sequence contains at least one unknown base")
```

Exactly one of these two blocks is executed.

4.4 More alternatives with elif

`elif` means “else if”. It checks another condition when previous conditions were false.

```
In [ ]: gc_percentage = 57.5

if gc_percentage >= 60:
    print("High GC")
elif gc_percentage >= 40:
    print("Moderate GC")
else:
    print("Low GC")
```

Python tests the branches from top to bottom and stops after the first true condition.

For 57.5 :

1. `57.5 >= 60` is false;
2. `57.5 >= 40` is true;
3. Python prints `Moderate GC` ;
4. the `else` block is skipped.

4.5 The order of conditions matters

This order is incorrect:

```
if gc_percentage >= 40:
    print("Moderate or high")
elif gc_percentage >= 60:
    print("High")
```

A value of 70 satisfies the first condition, so Python never reaches the more specific high-GC condition.

When thresholds overlap, test the more restrictive condition first.

Guided practice 3 — Classify sequence length

Use these rules:

- at least 1000 bases: `long` ;
- at least 100 but less than 1000: `medium` ;
- less than 100: `short` .

```
In [ ]: sequence_length = 480

if sequence_length >= 1000:
    category = "long"
elif sequence_length >= 100:
    category = "medium"
else:
    category = "short"

print("Length category:", category)
```

4.6 Nested conditions

An `if` block may contain another `if` . This is called nesting.

```
In [ ]: dna = "ATGCGTAA"

if len(dna) > 0:
    print("Sequence is not empty")

    if "N" in dna:
        print("Unknown bases detected")
    else:
        print("No unknown bases detected")
else:
    print("Sequence is empty")
```

Nesting is useful, but many levels become difficult to read. Use it only when the second decision logically depends on the first.

Part 5 — Validating one DNA sequence

For this introductory exercise, a valid DNA sequence:

- is not empty; and
- contains only `A` , `T` , `G` and `C` .

We can test every character later with a loop. For a single short string, a simple demonstration uses a set comparison, but sets have not yet been taught. We will therefore first check known unwanted symbols with string operations.

```
In [ ]: raw_dna = " atGcgtAa "
dna = raw_dna.strip().upper()

if len(dna) == 0:
    print("Invalid: the sequence is empty")
elif "N" in dna:
    print("Invalid: unknown base N detected")
```

```
else:  
    print("Basic check passed")
```

This is only a partial validation because symbols other than `N` might still occur. A loop will allow us to check every character properly.

Part 6 — Collections: storing several values

So far, one variable usually held one main value:

```
sequence = "ATGCGT"
```

Biological analyses commonly involve several samples or sequences. A **collection** holds multiple values under one name.

This lecture introduces:

- lists; and
- tuples.

Part 7 — Lists

7.1 Creating a list

A list uses square brackets. Items are separated by commas.

```
In [ ]: sequences = ["ATGC", "GCTA", "TTAA"]  
  
print(sequences)  
print(type(sequences))
```

A list may store strings, numbers or other Python values. Keeping similar types together usually makes the purpose clearer.

```
sample_names = ["sample_A", "sample_B", "sample_C"]  
read_counts = [1200, 850, 1640]
```

```
In [ ]: sample_names = ["sample_A", "sample_B", "sample_C"]  
read_counts = [1200, 850, 1640]  
  
print(sample_names)  
print(read_counts)
```

7.2 List length

`len()` returns the number of items in a list.

```
In [ ]: sample_names = ["sample_A", "sample_B", "sample_C"]  
  
print("Number of samples:", len(sample_names))
```

This differs from `len("sample_A")`, which counts characters in a string. The same function operates on different types in a sensible way.

7.3 Accessing list items

Lists use zero-based indexing, just like strings.

```
In [ ]: sample_names = ["sample_A", "sample_B", "sample_C"]

print("First sample:", sample_names[0])
print("Second sample:", sample_names[1])
print("Last sample:", sample_names[-1])
```

7.4 Slicing a list

List slices follow the same start-included, stop-excluded rule as string slices.

```
In [ ]: sample_names = ["sample_A", "sample_B", "sample_C", "sample_D"]

print(sample_names[:2])
print(sample_names[1:3])
print(sample_names[-2:])
```

The result of slicing a list is another list.

7.5 Lists are mutable

Mutable means that an object can be changed after it is created.

Replace an element by assigning a new value at its index.

```
In [ ]: sample_names = ["sample_A", "sample_B", "sample_C"]
print("Before:", sample_names)

sample_names[1] = "sample_B_corrected"
print("After:", sample_names)
```

The list itself was changed. This is different from strings, which are immutable.

7.6 Adding items with `append()`

`append()` adds one item to the end of a list.

```
In [ ]: sequences = ["ATGC", "GCTA"]
print("Before:", sequences)

sequences.append("TTAA")
print("After:", sequences)
```

`append()` modifies the existing list. It does not create a separate returned list.

A common mistake is:

```
sequences = sequences.append("TTAA")
```

After this statement, `sequences` becomes `None` because `append()` changes the list in place and returns no new list.

```
In [ ]: demonstration = ["ATGC", "GCTA"]
        returned_value = demonstration.append("TTAA")

        print("Modified list:", demonstration)
        print("Value returned by append:", returned_value)
```

7.7 Inserting at a specific position

`insert(index, value)` adds an item at a selected position.

```
In [ ]: sample_names = ["sample_A", "sample_C"]
        sample_names.insert(1, "sample_B")

        print(sample_names)
```

Existing items at and after the insertion position move to the right.

7.8 Removing items

`remove(value)` removes the first matching value.

```
In [ ]: sample_names = ["sample_A", "failed_sample", "sample_C"]
        sample_names.remove("failed_sample")

        print(sample_names)
```

`pop(index)` removes and returns an item at an index. Without an index, it removes the last item.

```
In [ ]: sample_names = ["sample_A", "sample_B", "sample_C"]
        removed_sample = sample_names.pop(1)

        print("Removed:", removed_sample)
        print("Remaining:", sample_names)
```

Use `remove()` when you know the value. Use `pop()` when you know the position and may need the removed value.

7.9 Checking list membership

```
In [ ]: sample_names = ["sample_A", "sample_B", "sample_C"]

        print("sample_B" in sample_names)
        print("sample_X" not in sample_names)
```

Guided practice 4 — Modify a sample list

Starting with the supplied list:

1. replace `sample_B_wrong` with `sample_B` ;
2. append `sample_D` ;
3. insert `sample_0` at the beginning;
4. remove `failed_sample` ; and
5. print the final list and its length.

```
In [ ]: samples = ["sample_A", "sample_B_wrong", "failed_sample", "sample_C"]

samples[1] = "sample_B"
samples.append("sample_D")
samples.insert(0, "sample_0")
samples.remove("failed_sample")

print(samples)
print("Number of samples:", len(samples))
```

Part 8 — Tuples

8.1 Creating a tuple

A tuple commonly uses parentheses.

```
In [ ]: allowed_bases = ("A", "T", "G", "C")

print(allowed_bases)
print(type(allowed_bases))
```

Tuples support indexing, slicing, `len()` and membership tests.

```
In [ ]: allowed_bases = ("A", "T", "G", "C")

print(allowed_bases[0])
print(allowed_bases[-1])
print(len(allowed_bases))
print("N" in allowed_bases)
```

8.2 Tuples are immutable

Immutable means that items cannot be replaced, appended or removed after the tuple is created.

The following would cause a `TypeError` :

```
allowed_bases[0] = "U"
```

A tuple is useful when a collection represents fixed information that should not change accidentally.

8.3 List versus tuple

Feature	List	Tuple
Syntax	<code>["A", "T"]</code>	<code>("A", "T")</code>
Can replace items?	Yes	No
Can append items?	Yes	No
Typical use	Changing collection	Fixed collection

For example:

- a growing list of sample names can be a list;
- the fixed set of four expected DNA bases can be a tuple.

8.4 A one-item tuple needs a comma

```
not_a_tuple = ("A")
one_item_tuple = ("A",)
```

```
In [ ]: not_a_tuple = ("A")
        one_item_tuple = ("A",)

        print(type(not_a_tuple))
        print(type(one_item_tuple))
```

The comma, not the parentheses alone, creates the one-item tuple.

Part 9 — Why loops are needed

Suppose we want the length of three sequences.

Without a loop:

```
print(len("ATGC"))
print(len("GCTAAG"))
print(len("TTAA"))
```

This repeats nearly identical code. A loop describes the repeated action once.

Part 10 — for loops

10.1 Basic structure

```
for item in collection:
    instruction_using_item
```

Read it as:

For each item in the collection, run the indented instruction.

```
In [ ]: sequences = ["ATGC", "GCTAAG", "TTAA"]
```

```
for sequence in sequences:
    print(sequence)
```

Step by step:

1. `sequence` refers to `"ATGC"` ; the body runs.
2. `sequence` refers to `"GCTAAG"` ; the body runs.
3. `sequence` refers to `"TTAA"` ; the body runs.
4. No items remain; the loop ends.

The singular loop-variable name `sequence` and plural list name `sequences` help communicate their different roles.

10.2 Process each sequence

```
In [ ]: sequences = ["ATGC", "GCTAAG", "TTAA"]

for sequence in sequences:
    length = len(sequence)
    print(sequence, "has length", length)
```

The two indented statements belong to the loop and therefore run for every sequence.

10.3 Code after a loop

```
In [ ]: sequences = ["ATGC", "GCTAAG", "TTAA"]

for sequence in sequences:
    print("Processing:", sequence)

print("All sequences processed")
```

The final line is not indented, so it runs once after the loop finishes.

10.4 Loop over a string

A string is a sequence of characters, so a `for` loop can visit each character.

```
In [ ]: dna = "ATGC"

for base in dna:
    print(base)
```

This provides a complete way to validate every base.

```
In [ ]: dna = "ATGCXTA"
allowed_bases = ("A", "T", "G", "C")

for base in dna:
    if base in allowed_bases:
        print(base, "is valid")
    else:
        print(base, "is invalid")
```

The `if` statement is inside the loop, so each base is checked separately.

10.5 Building a count with an accumulator

An accumulator is a variable updated repeatedly to build a result.

```
In [ ]: dna = "ATGCGTAA"
        g_count = 0

        for base in dna:
            if base == "G":
                g_count = g_count + 1

        print("G count:", g_count)
```

Trace the value:

```
Start: g_count = 0
See A: unchanged
See T: unchanged
See G: g_count becomes 1
See C: unchanged
See G: g_count becomes 2
...
```

The shorter update `g_count += 1` means the same as `g_count = g_count + 1`.

```
In [ ]: dna = "ATGCGTAA"
        g_count = 0

        for base in dna:
            if base == "G":
                g_count += 1

        print("G count:", g_count)
```

Guided practice 5 — Count unknown bases

Count the number of `N` characters without using `.count()`.

```
In [ ]: dna = "ATGCNNATN"
        n_count = 0

        for base in dna:
            if base == "N":
                n_count += 1

        print("N count:", n_count)
```

Part 11 — `range()`

`range()` generates a sequence of integers, commonly used with a `for` loop.

11.1 range(stop)

The stop value is excluded.

```
In [ ]: for number in range(5):  
        print(number)
```

The values are 0, 1, 2, 3 and 4.

11.2 range(start, stop)

```
In [ ]: for number in range(1, 6):  
        print(number)
```

The values are 1 through 5 because 6 is excluded.

11.3 range(start, stop, step)

```
In [ ]: for number in range(0, 10, 2):  
        print(number)
```

The step of 2 generates 0, 2, 4, 6 and 8.

11.4 Use indices when positions matter

```
In [ ]: dna = "ATGCGT"  
  
for index in range(len(dna)):  
    base = dna[index]  
    print(index, base)
```

`range(len(dna))` generates every valid positive index of the string.

Part 12 — enumerate()

`enumerate()` provides both an index and the corresponding item.

```
In [ ]: dna = "ATGCGT"  
  
for index, base in enumerate(dna):  
    print(index, base)
```

This is often clearer than manually indexing with `range(len(dna))`.

To display positions beginning at 1 for a human reader:

```
In [ ]: dna = "ATGCGT"
```

```
for position, base in enumerate(dna, start=1):
    print(position, base)
```

The biological display position begins at 1, although the underlying Python string remains zero-indexed.

Guided practice 6 — Report motif positions

Report every position containing `G`. Display human-readable positions beginning at 1.

```
In [ ]: dna = "ATGCGTAGG"

for position, base in enumerate(dna, start=1):
    if base == "G":
        print("G found at position", position)
```

Part 13 — while loops

A `for` loop is appropriate when processing items in a collection. A `while` loop repeats while a condition remains true.

```
while condition:
    repeated_instruction
```

```
In [ ]: count = 1

while count <= 5:
    print(count)
    count += 1
```

The update `count += 1` is essential. Without it, `count <= 5` remains true and the loop may continue indefinitely.

If a notebook cell runs indefinitely, interrupt the kernel. In Jupyter, select **Kernel** → **Interrupt Kernel** or use the stop button.

13.1 Trace a while loop

```
count = 1 → condition true → print 1 → count becomes 2
count = 2 → condition true → print 2 → count becomes 3
...
count = 5 → condition true → print 5 → count becomes 6
count = 6 → condition false → loop stops
```

13.2 A controlled trimming example

Remove leading `N` characters one at a time.

```
In [ ]: dna = "NNNATGCGT"

while dna.startswith("N"):
    dna = dna[1:]

print("Trimmed sequence:", dna)
```

Each repetition removes the first character. The loop stops when the sequence no longer begins with `N`.

In normal code, `lstrip("N")` could perform this particular task more directly. The loop is shown to demonstrate repeated condition-controlled change.

Part 14 — `break` and `continue`

These statements alter normal loop flow. Use them sparingly and only when they make the logic clearer.

14.1 `break` stops a loop

```
In [ ]: dna = "ATGCXTAA"
allowed_bases = ("A", "T", "G", "C")

for position, base in enumerate(dna, start=1):
    if base not in allowed_bases:
        print("Invalid base", base, "at position", position)
        break
```

After the first invalid base is found, no further positions are checked.

14.2 `continue` skips the remainder of one iteration

```
In [ ]: dna = "ATGCNTAA"

for base in dna:
    if base == "N":
        continue
    print("Identified base:", base)
```

When the base is `N`, Python skips the print statement and moves to the next character.

Part 15 — Integrated practical: process several sequences

We will combine:

- a list;
- a `for` loop;

- string cleaning;
- conditions;
- nucleotide counting; and
- formatted output.

The data are deliberately small so that results can be checked manually.

```
In [ ]: sequences = [
    "ATGCGTAA",
    "ggccaatt",
    "ATGCNNTA",
    ""
]

for raw_sequence in sequences:
    sequence = raw_sequence.strip().upper()

    print("-" * 35)
    print("Sequence:", repr(sequence))

    if len(sequence) == 0:
        print("Status: empty sequence")
    elif "N" in sequence:
        print("Status: contains unknown bases")
    else:
        length = len(sequence)
        gc_count = sequence.count("G") + sequence.count("C")
        gc_percentage = (gc_count / length) * 100

        print("Status: suitable for basic GC calculation")
        print("Length:", length)
        print(f"GC percentage: {gc_percentage:.2f}%")
```

15.1 Why the empty check comes first

An empty sequence has length zero. Dividing by zero would cause a `ZeroDivisionError`. The first condition prevents the calculation for empty input.

15.2 Why the unknown-base check comes second

The program applies a stated rule: basic GC percentage is reported only for sequences without `N`. Another analysis might use a different policy. Good scientific code makes such decisions explicit.

Guided practice 7 — Add a GC category

Modify the integrated analysis so that valid sequences are classified as:

- `high GC` when GC percentage is at least 60;
- `moderate GC` when it is at least 40 but below 60; and
- `low GC` when it is below 40.

Attempt the activity before opening the solution.

```
In [ ]: practice_sequences = ["GCGCGCAA", "ATGCGTAA", "ATATTTAA"]  
  
# Write or adapt your loop here.
```

► Open one possible solution

Part 16 — Common errors and how to reason about them

16.1 Missing colon

Incorrect:

```
if gc_percentage >= 50  
    print("At least 50%")
```

Correct:

```
if gc_percentage >= 50:  
    print("At least 50%")
```

16.2 Incorrect indentation

Incorrect:

```
for base in dna:  
print(base)
```

Correct:

```
for base in dna:  
    print(base)
```

Indentation tells Python which instructions belong to the block.

16.3 Using `=` instead of `==`

Incorrect comparison:

```
if base = "G":
```

Correct comparison:

```
if base == "G":
```

16.4 Modifying a list while looping over it

Removing items from the same list currently being traversed can skip elements unexpectedly.

Avoid this pattern for now:

```

for sample in samples:
    if sample == "failed":
        samples.remove(sample)

```

Beginners should instead build a new list of accepted values. This will be practised further in later work.

16.5 Infinite while loop

Incorrect:

```

count = 1
while count <= 5:
    print(count)

```

count never changes, so the condition remains true.

Correct:

```

count = 1
while count <= 5:
    print(count)
    count += 1

```

16.6 Index out of range

The valid indices of a three-item list are 0, 1 and 2.

```

In [ ]: samples = ["A", "B", "C"]

try:
    print(samples[3])
except IndexError as error:
    print(type(error).__name__ + ":", error)

```

Use `len(samples)` to inspect the number of items. The final item is `samples[-1]`.

Part 17 — AI-assisted challenge

Ask an AI assistant to explain this program line by line:

```

sequences = ["ATGC", "ATGN", "GGCC"]
valid_bases = ("A", "T", "G", "C")

for sequence in sequences:
    is_valid = True

    for base in sequence:
        if base not in valid_bases:
            is_valid = False
            break

    if is_valid:

```

```
        print(sequence, "valid")
    else:
        print(sequence, "invalid")
```

Then verify the explanation yourself:

1. Why is `is_valid` set to `True` at the start of every outer-loop iteration?
2. What makes it become `False` ?
3. Which loop does `break` stop?
4. What happens for `GGCC` ?
5. What would go wrong if `is_valid = True` appeared before the outer loop?

Do not accept an explanation until you can trace the variables manually.

Part 18 — Independent practice

Activity A — Quality decision

For a sample with a read count and percentage of ambiguous bases, report `PASS` only when read count is at least 1000 and ambiguous percentage is no more than 2.

Activity B — List editing

Create a list of four sample names. Replace one name, append another sample, remove one failed sample, and print the final number of samples.

Activity C — Nucleotide counts with a loop

For `ATGCGTAA`, count A, T, G and C using a loop and four accumulator variables. Do not use `.count()`.

Activity D — Validate every character

Check each character of `ATGCXTA`. Report the human-readable position and symbol of every invalid character.

Activity E — Process multiple sequences

For a list of at least four sequences, print sequence, length, GC percentage and GC category. Handle an empty sequence without causing division by zero.

Part 19 — Final review

Complete these statements without looking back:

1. `=` is used to `__`.

2. `==` is used to ____.
3. A condition produces ____.
4. `and` is true when ____.
5. `or` is true when ____.
6. `if` is used to ____.
7. `elif` is used when ____.
8. `else` runs when ____.
9. A list is ____.
10. A tuple differs from a list because ____.
11. `append()` ____.
12. A `for` loop ____.
13. A `while` loop ____.
14. `range(5)` produces ____.
15. `enumerate()` provides ____.
16. An accumulator ____.

If you can trace the integrated sequence-processing loop and explain why each branch runs, you have achieved the main goals of Lecture 4.

Compact reference sheet

Comparisons

```
x == y
x != y
x > y
x >= y
x < y
x <= y
```

Boolean logic

```
condition_a and condition_b
condition_a or condition_b
not condition_a
```

Decision

```
if condition_1:
    instruction
elif condition_2:
    alternative
else:
    fallback
```

List

```
items = ["A", "B", "C"]
items[0]
items[1] = "B_corrected"
items.append("D")
items.insert(0, "start")
items.remove("C")
```

Tuple

```
fixed_items = ("A", "T", "G", "C")

# For loop
for item in items:
    print(item)

# Position and value
for position, item in enumerate(items, start=1):
    print(position, item)

# Controlled while loop
count = 1
while count <= 5:
    print(count)
    count += 1
```

Before submitting your notebook

- [] Restart the kernel and run all cells in order.
- [] Confirm that all outputs are reasonable.
- [] Complete the guided and independent activities.
- [] Do not leave unexplained error messages.
- [] Ensure every `if`, `for` and `while` block is correctly indented.
- [] Confirm that every `while` loop can eventually stop.
- [] Save the notebook with a meaningful filename.
- [] Commit and push it to your `SBI811-Exercises` repository when instructed.